

Slide 1 — Titre et intention

Bonjour, je m'appelle Xabi Martinez. Cette présentation CCP s'appuie sur mon projet PokeRNCP, un backend de Pokédex développé en Rust. Mon objectif est de démontrer, de façon claire et concrète, la maîtrise des compétences attendues au RNCP pour la partie back-end : modélisation relationnelle, exposition d'une API REST sécurisée, tests, qualité et déploiement conteneurisé. Je vais expliquer les choix, montrer comment ils se traduisent dans le code et la base, et mettre en perspective les limites et les améliorations possibles. L'idée n'est pas de tout montrer, mais de présenter une architecture qu'on peut expliquer, maintenir, et faire évoluer. À la fin, vous aurez une vision précise de ce que fait PokeRNCP et pourquoi il a été conçu ainsi.

Slide 2 — Sommaire

Voici le fil de la présentation :

1. Contexte rapide et objectifs du projet.
 2. Explication concrète de ce que fait PokeRNCP.
 3. Modélisation et gestion des données.
 4. Sécurité et flux clés.
 5. Qualité : tests, performance, déploiement.
 6. compétences couvertes et conclusion. Ce sommaire vous donne la vision d'ensemble avant de plonger dans les détails.
-

Slide 3 — Contexte du projet : Introduction

PokeRNCP est une API REST de type Pokédex. Elle permet à tout utilisateur de créer un compte, de s'authentifier puis de collectionner les Pokémon des six premières générations. L'objectif n'est pas de couvrir tout l'univers Pokémon, mais de proposer un socle propre et fiable : des endpoints clairs, une modélisation simple, une sécurité crédible et une base prête à évoluer , par exemple l'ajout de nouvelles générations.

Slide 4 — Contexte du projet : Équipe

Je m'appelle Xabi Martinez, 32 ans, basé au Pays Basque, cohort Holberton C#24. Ce projet a été mené seul du début à la fin : conception, développement, tests, documentation et conteneurisation. Travailler en solo m'a obligé à être structuré sur l'organisation (prioriser un MVP, puis lister les améliorations), à gérer mon temps et à documenter chaque décision pour conserver de la clarté. Cela illustre ma capacité à être autonome tout en gardant une démarche professionnelle.

Slide 5 — Contexte du projet : Contraintes

Trois contraintes principales ont guidé la réalisation :

- Temps : concilier apprentissage de Rust et production d'un livrable propre, ainsi que mon emploi.

- Organisation : structurer le travail seul (MVP puis itérations ciblées) sans dispersion fonctionnelle.
 - Compétences : montée en compétence sur Rust ainsi qu'en pratiques backend . Ces contraintes ont motivé des choix de simplicité et de robustesse plutôt que d'exhaustivité fonctionnelle.
-

Slide 6 — Contexte du projet : Utilisation

Cas d'usage principaux couverts par l'API :

- CRUD de compte : inscription, login, rafraîchissement de session, logout.
 - Capture de Pokémon : association utilisateur, Pokémon sans doublon pour l'idempotence.
 - Consultation des données : liste enrichie avec indicateur de capture, informations de base sur chaque Pokémon.
 - Recherche ciblée : possibilité de retrouver un Pokémon spécifique par son nom. Ces usages simples valident la structure, la sécurité de base et l'intégrité des données.
-

Slide 7 — Gestion du projet : Outils

Pour mener le projet efficacement, j'ai utilisé un ensemble d'outils complémentaires : VS Code pour l'édition de code avec rust-analyzer et Clippy, GitHub pour le versionnement et la traçabilité, GitHub Copilot pour accélérer certains brouillons tout en relisant systématiquement, Postman pour les tests exploratoires d'API, pgAdmin4 pour l'inspection du schéma et des requêtes, Docker / docker-compose pour des environnements reproductibles, ainsi que Cargo et SQLx pour la gestion des dépendances et la validation statique des requêtes. Cet écosystème m'a permis d'itérer rapidement, de sécuriser les évolutions et de garder une organisation claire.

Slide 8 — Architecture globale

Vue d'ensemble : un client (web ou mobile) communique en HTTP/REST avec l'API Axum (Rust). Cette API s'appuie sur PostgreSQL pour la persistance et sur SQLx pour des requêtes typées et sûres. La couche d'authentification émet et valide un duo de JWT (access court, refresh plus long) avec cookies HttpOnly. Un build Docker multi-stage produit une image légère pour le déploiement reproductible. Les échanges principaux : requêtes REST, requêtes SQL paramétrées, génération/validation des tokens, conteneurisation de l'application. Cette architecture sépare nettement interface, logique métier, gestion des sessions et stockage, ce qui rend le backend explicable, testable et extensible.

Slide 9 — Méthode Merise

Avant de montrer les schémas, je replace la démarche Merise. On part du métier et on modélise le réel avec le MCD : ici, deux entités « Users » et « Pokémon », reliées par l'association « User_Pokemon / CAPTURE » qui formalise la relation $N \leftrightarrow N$ (un utilisateur peut capturer plusieurs Pokémon, et chaque Pokémon peut être capturé par plusieurs utilisateurs).

On pose les cardinalités et les règles de gestion, puis les contraintes : l'unicité du couple (utilisateur, pokémon) pour éviter les doublons, et une suppression en cascade côté utilisateur pour préserver l'intégrité.

On identifie aussi un attribut dérivé « caught » : il n'est pas stocké, il se déduit de l'existence d'une capture (il reste donc hors MCD et sera calculé au niveau logique/lecture). Cette méthode produit un modèle stable et normalisé, indépendant de la technique.

Slide 10 — MCD

Voici le Modèle Conceptuel de Données issue de la méthode Merise. On y voit les entités « Users » et « Pokémon » définies sans considérations techniques : juste le métier (un utilisateur, un Pokémon). L'association « CAPTURE » (User_Pokemon) porte la relation $N \leftrightarrow N$ et encapsule la règle « un couple (utilisateur, pokémon) est unique ». Le MCD ne contient ni types SQL, ni clés primaires physiques ; il formalise seulement les cardinalités et contraintes fonctionnelles. L'attribut dérivé « caught » n'apparaît pas car il se déduit d'une capture existante — il reste calculé au niveau logique. Ce diagramme garantit une base stable et normalisée avant toute traduction technique.

Slide 11 — MPD

Passage au Modèle Physique : chaque entité devient une table relationnelle. « Users » devient `users` avec un identifiant (`id`) généré, un pseudo unique, un email éventuellement unique et un mot de passe haché. « Pokémon » devient `pokemon` avec son nom unique et ses attributs descriptifs. L'association CAPTURE devient la table `user_pokemon` avec deux colonnes de liaison (`user_id`, `pokemon_id`), plus les attributs propres. La contrainte fonctionnelle d'unicité se traduit par `UNIQUE(user_id, pokemon_id)` (index composite utilisé aussi pour l'idempotence), et la suppression en cascade sur `user_id` assure l'intégrité référentielle. Les index simples sur `user_pokemon.user_id` et `user_pokemon.pokemon_id` accélèrent les parcours fréquents. L'attribut dérivé « caught » n'est pas stocké : il est obtenu via une sous-requête `EXISTS` lors de la lecture. Ce MPD est ainsi une traduction fidèle du MCD, prête pour l'exécution efficace.

Slide 12 — Initialisation DB

Ici nous pouvons voir la migration initiale qui pose le socle de la base: activation de `pgcrypto` pour générer des identifiants, création de `users` (identifiant, nom unique, email optionnelle unique, mot de passe, date de création), de `pokemon` (nom unique, types, statistiques de base) et de l'association `user_pokemon` (lien utilisateur↔pokémon, surnom, date de découverte) avec une contrainte d'unicité sur le couple (utilisateur, pokémon) pour empêcher les doublons et une suppression en cascade côté utilisateur. Deux index simples accélèrent les parcours fréquents: un sur `user_id` pour lister/compter rapidement les captures d'un utilisateur, et un sur `pokemon_id` pour retrouver efficacement tous les utilisateurs ayant capturé un Pokémon donné; la contrainte `UNIQUE(user_id, pokemon_id)` crée en plus un index unique composite, utilisé notamment pour la vérification « déjà capturé » sans scan complet.

Slide 13 — Migrations complémentaires

Après le changement de la source JSON, trois migrations incrémentales ont ajusté le schéma : La première ajoute le numéro Pokédex et l'URL d'image ; la deuxième enrichit avec la taille et le poids, et introduit temporairement `weaknesses` ; la troisième retire `weaknesses` et ajoute une `description` textuelle. Chaque migration est versionnée et réversible (up/down), ce qui permet de tracer et d'adapter le modèle sans casser le contrat de l'API.

Slide 14 — Sécurité de l'API

La sécurité s'appuie sur un duo de JWT avec un access token de courte durée et un refresh de plus longue durée, le tout porté par des cookies `HttpOnly` configurés avec `SameSite` (`Lax`/`Strict` selon contexte). Les mots de passe sont protégés par `Argon2`. Des middlewares `Axum` centralisent l'extraction/validation des jetons et la gestion des erreurs pour éviter la duplication de logique sensible. Contre le CSRF, la combinaison cookies `HttpOnly` + stratégie `SameSite` et, si nécessaire, un en-tête anti-CSRF côté client, limite les requêtes inter-sites non désirées sans dégrader l'ergonomie.

Slide 15 — Flux applicatifs

Login : validation des identifiants (comparaison au secret protégé par `Argon2`) puis émission d'un duo de jetons (access court, refresh long en cookie `HttpOnly`). Refresh automatique : un endpoint dédié renouvelle l'access si le refresh est valide. Listing Pokémon : renvoie la liste enrichie, avec l'indicateur « capturé » calculé quand l'utilisateur est connecté. Capture (idempotente) : l'opération ignore les doublons grâce à l'unicité (utilisateur, pokémon). Vérifications & permissions : des middlewares `Axum` valident le token, extraient l'identité et filtrent l'accès aux routes protégées.

Slide 16 — Extraits de code

Deux extraits tirés de `backend/src/handlers/pokemon.rs` : `catch` vérifie d'abord que le Pokémon existe (recherche par nom, sinon « Pokémon introuvable »), puis insère dans `user_pokemon` avec `ON CONFLICT (user_id, pokemon_id) DO NOTHING` pour garantir l'idempotence. `list_all` renvoie la liste des Pokémon en calculant l'indicateur « caught » via une sous-requête `EXISTS` (aucun doublon stocké). Ces extraits illustrent une écriture sûre côté capture et une lecture enrichie côté liste.

Cette slide montre deux extraits de code : d'abord la fonction `catch` vérifie d'abord que le Pokémon existe, puis insère dans `user_pokemon` avec un `ON CONFLICT (user_id, pokemon_id) DO NOTHING` pour garantir l'idempotence et la fonction `list_all`, qui renvoie la liste des Pokémon en calculant l'indicateur « caught » via une sous-requête `EXISTS`. Ces deux morceaux illustrent le cœur du flux : lecture enrichie côté liste, écriture sûre côté capture.

Slide 17 — Performance et optimisations

Rust + Tokio offrent un modèle asynchrone performant et prévisible ; les requêtes SQLx sont vérifiées à la compilation, réduisant les erreurs au runtime. Au démarrage, une logique de retry attend la disponibilité de la base avant d'exposer l'API. L'idempotence côté capture évite les écritures redondantes et les limites sur la recherche bornent la charge. Enfin, l'état « caught » est calculé dynamiquement via `EXISTS`, ce qui évite les duplications et garde la lecture efficace.

Slide 18 — Qualité et tests

La qualité repose sur des tests d'intégration (via `reqwest`) exécutés rapidement en parallèle avec `nextest`, le linter de `rust` (`clippy`) pour rester aligné avec les bonnes pratiques du langage, et une séparation propre du code pour limiter les couplages. La documentation accompagne les choix afin de faciliter la revue, l'onboarding et l'évolution du projet sans régressions.

Slide 19 — Tests (extraits)

Deux captures de code : d'abord un qui vérifie la connectivité à la base par un `SELECT 1` simple. Et un deuxième qui couvre le parcours de création d'utilisateur avec succès (201) puis la tentative de doublon qui renvoie un `409 Conflict` pour vérifier que l'unicité est bien respecté. Ces tests illustrent la stratégie : valider l'infrastructure de base et sécuriser les cas métier critiques.

Slide 20 — Tests

Cette slide est une capture d'écran de l'exécution de `cargo nextest run`. Elle montre l'exécution parallèle des tests d'intégration, le résumé (succès/échecs), les durées par test et la sortie colorée. L'objectif est d'illustrer la rapidité de feedback et la stabilité de la suite de tests dans ce projet.

Slide 21 — Crates utilisés

Cette slide rassemble deux des piliers techniques du projet. D'un côté, Tokio, le runtime asynchrone qui donne son rythme à l'application : il orchestre des tâches concurrentes sans bloquer, s'appuie sur les files d'événements du système et met à profit le modèle de propriété de Rust pour conjuguer performance et fiabilité. Concrètement, c'est lui qui permet à l'API de rester réactive sous charge, de traiter plusieurs requêtes en parallèle et d'exploiter efficacement les ressources de la machine.

De l'autre, Argon2 protège ce qui doit l'être : les mots de passe. Son approche « memory-hard » décourage les attaques massives et son variant Argon2id, utilisé par défaut, combine le meilleur des approches i et d pour résister à la fois aux canaux auxiliaires et aux tentatives par force brute. En pratique, cela signifie que l'authentification repose sur des fondations cryptographiques solides, sans compromis sur la sécurité, tout en restant entièrement implémentée en Rust.

Slide 22 — Déploiement professionnel

Le projet est conditionné pour un déploiement propre : un Docker multi-stage compile Rust en image finale légère, `docker-compose` orchestre l'API et la base PostgreSQL (réseau/volumes dédiés), les variables d'environnement pilotent la configuration sans exposer de secrets dans le dépôt, et l'ensemble reste portable entre machines. Pour des démonstrations externes, un tunnel HTTPS peut être activé afin d'exposer l'API de façon sécurisée sans ouvrir le pare-feu.

Slide 23 — Compétences RNCP couvertes

Cette réalisation couvre les attendus du RNCP de bout en bout. Côté données, j'ai formalisé le domaine avec Merise, puis matérialisé un SQL relationnel simple et robuste pour garantir intégrité et évolutivité. Côté application, j'ai développé une API REST claire et sécurisée. La sécurité est traitée de manière systémique : mots de passe hachés avec Argon2, gestion de session par JWT et transport via cookies HttpOnly configurés. Enfin, la qualité d'exécution s'appuie sur des tests d'intégration automatisés et sur une conteneurisation prête au déploiement : images Rust multi-stage légères, orchestration `docker-compose` et variables d'environnement pour piloter la configuration. L'ensemble démontre la capacité à concevoir, implémenter, sécuriser et livrer un service professionnel.

Slide 24 — Conclusion

En conclusion, le travail fourni aboutit à un projet complet et opérationnel : un système de connexion sécurisé et un système de capture dynamique, le tout porté par une architecture claire. Ce parcours a été une vraie étape d'apprentissage, avec notamment la découverte d'Axum et une approche résolument professionnelle : code structuré, tests automatisés, conteneurisation et déploiement soignés. Pour la suite, je souhaite approfondir la sécurité, mettre en place une CI/CD et ajouter progressivement des features utiles. Merci pour votre attention — je suis disponible pour vos questions.